Your understanding of shared versus exclusive locking and Condition is broadly correct. The biggest corrections are that synchronized **can** protect code across different objects when they share the same monitor, Semaphore is not a Lock implementation, and StampedLock optimistic reading is different from version-based optimistic locking in databases.

Java Locks and Conditions

1. What is a critical section?

A critical section is code that accesses shared mutable state and must be protected from unsafe concurrent access.

```
balance = balance - amount;
```

The operation above is not a single atomic action. Conceptually, it involves:

```
Read balance
    ↓
Calculate new balance
    ↓
Write new balance
```

If multiple threads execute these steps concurrently, updates can be lost.

A lock creates a relationship like:

```
Acquire lock
    ↓
Access shared state
    ↓
Release lock
```

The guiding principle is:

> Threads must use the same lock to protect the same shared invariant.

---

2. Can synchronized coordinate different objects?

You said:

> No matter whether it is the same object or different objects, only one thread should enter the critical section. synchronized will not work.

This needs qualification.

A synchronized instance method locks the current object:

```
public synchronized void process() {
    // Lock is this
}
```

This is equivalent to:

```
public void process() {
    synchronized (this) {
        // Critical section
    }
}
```

If two threads call the method on two different instances, they acquire different monitors:

```
Service first = new Service();
Service second = new Service();

first.process();  // Locks first
second.process(); // Locks second
```

Both can execute concurrently because:

```
Thread 1 → locks first object
Thread 2 → locks second object
```

However, synchronized **can protect across different instances** if those instances synchronize on the same shared monitor:

```
public class Service {

    private static final Object GLOBAL_LOCK = new Object();

    public void process() {
        synchronized (GLOBAL_LOCK) {
            // Only one thread across all Service instances
        }
    }
}
```

Now:

```
first.process()  ─┐
                  ├── Both compete for GLOBAL_LOCK
second.process() ─┘
```

A static synchronized method also uses one class-level monitor:

```
public static synchronized void process() {
    // Locks Service.class
```

```
}
```

Equivalent to:

```
public static void process() {
    synchronized (Service.class) {
        // Critical section
    }
}
```

Correct rule

It is not whether the application uses synchronized or an explicit Lock.

The real question is:

> Are all participating threads acquiring the same monitor or lock instance?

Both of these can fail if every object has its own lock:

```
synchronized (this) {
}
```

```
private final Lock lock = new ReentrantLock();
```

Both can coordinate globally if every object uses one shared lock:

```
private static final Object MONITOR = new Object();
```

```
private static final Lock LOCK = new ReentrantLock();
```

3. Main Java locking and coordination tools

The following are related concurrency utilities, but they are not all implementations of the Lock interface:

```
ReentrantLock
    → Exclusive Lock implementation

ReentrantReadWriteLock
    → ReadWriteLock implementation

StampedLock
    → Separate locking utility
    → Does not implement Lock or ReadWriteLock

 Semaphore
     → Permit-based synchronizer
```

```
13      → Does not implement Lock
```

The primary lock-related interfaces are:

```
1  Lock
2  ReadWriteLock
3  Condition
```

4. ReentrantLock

ReentrantLock is an explicit mutual-exclusion lock.

```
1  Lock lock = new ReentrantLock();
```

Only one thread can hold it at a time:

```
1  lock.lock();
2
3  try {
4      accessSharedState();
5  } finally {
6      lock.unlock();
7  }
```

The finally block is essential. Otherwise, an exception can leave the lock permanently held.

Why is it called reentrant?

A lock is reentrant when the thread currently holding it can acquire it again.

```
1  private final Lock lock = new ReentrantLock();
2
3  public void outer() {
4      lock.lock();
5
6      try {
7          inner();
8      } finally {
9          lock.unlock();
10      }
11  }
12
13  private void inner() {
14      lock.lock();
15
```

```
16      try {
17          performWork();
18      } finally {
19          lock.unlock();
20      }
21  }
```

The same thread acquires the lock twice. It must release it twice.

Conceptually:

```
1  First lock()  → hold count = 1
2  Second lock() → hold count = 2
3  First unlock  → hold count = 1
4  Second unlock → hold count = 0
```

Java's intrinsic monitor used by synchronized is also reentrant.

---

5. ReentrantLock versus synchronized

Both provide exclusive mutual exclusion and memory-visibility guarantees.

ReentrantLock adds capabilities such as:

Interruptible acquisition

```
1  lock.lockInterruptibly();
```

A thread waiting for the lock can respond to interruption.

Non-blocking attempt

```
1  if (lock.tryLock()) {
2      try {
3          performWork();
4      } finally {
5          lock.unlock();
6      }
7  }
```

Timed acquisition

```
1  if (lock.tryLock(2, TimeUnit.SECONDS)) {
2      try {
3          performWork();
4      } finally {
```

```
5           lock.unlock();
6       }
7   }
```

Multiple conditions

```
1   Condition notEmpty = lock.newCondition();
2   Condition notFull = lock.newCondition();
```

Optional fairness

```
1   Lock lock = new ReentrantLock(true);
```

A fair lock generally favors threads that have waited longer, but fairness can reduce throughput. It also does not provide an absolute real-time scheduling guarantee.

Use synchronized when basic mutual exclusion is sufficient. Use ReentrantLock when the additional acquisition or condition capabilities are genuinely needed.

---

6. ReadWriteLock

A ReadWriteLock exposes two related locks:

```
1   ReadWriteLock readWriteLock =
2       new ReentrantReadWriteLock();
3
4   Lock readLock = readWriteLock.readLock();
5   Lock writeLock = readWriteLock.writeLock();
```

The two lock modes are:

```
1   Read lock  → Shared lock
2   Write lock → Exclusive lock
```

---

7. Shared read lock

Multiple threads can hold the read lock concurrently:

```
1   Thread 1 → read lock acquired
2   Thread 2 → read lock acquired
3   Thread 3 → read lock acquired
```

This is allowed because readers are expected not to modify the protected state.

Example:

```
1  private final ReadWriteLock lock =
2      new ReentrantReadWriteLock();
3
4  private final Map<String, String> cache =
5      new HashMap<>();
6
7  public String get(String key) {
8      lock.readLock().lock();
9
10      try {
11          return cache.get(key);
12      } finally {
13          lock.readLock().unlock();
14      }
15  }
```

Multiple calls to get() can execute concurrently.

---

8. Exclusive write lock

Only one thread can hold the write lock:

```
1  public void put(String key, String value) {
2      lock.writeLock().lock();
3
4      try {
5          cache.put(key, value);
6      } finally {
7          lock.writeLock().unlock();
8      }
9  }
```

When the write lock is held:

- No other thread can obtain the write lock
- No other thread can obtain the read lock

When one or more read locks are held:

- Other readers may acquire the read lock
- A writer must normally wait

Compatibility model:

```
Held lock    Requested read    Requested write
Read         Allowed           Blocked
Write        Blocked           Blocked
None         Allowed           Allowed
```

“Allowed” does not mean immediate execution is guaranteed, since scheduling and fairness policies may affect acquisition.

---

9. When is ReadWriteLock useful?

A read-write lock may help when:

- Reads greatly outnumber writes
- Protected operations are not extremely short
- There is meaningful contention
- Concurrent reads are safe
- The overhead of read-write locking is lower than the concurrency benefit

It may not help when:

- The critical section is tiny
- There is little contention
- Write operations are frequent
- The data structure already provides appropriate concurrency
- Lock-management overhead exceeds the benefit

Therefore, do not assume:

```
Many reads → ReadWriteLock is automatically faster
```

Measure it under realistic load.

For maps, a ConcurrentHashMap may be more suitable than manually locking a HashMap, depending on the required compound operations and invariants.

---

10. Lock downgrade and upgrade

Downgrading

A thread can acquire the read lock while already holding the write lock, then release the write lock.

Conceptually:

```
Hold write lock
    ↓
Acquire read lock
    ↓
Release write lock
    ↓
Continue holding read lock
```

This is called lock downgrading.

Upgrading

Trying to move directly from a read lock to a write lock is dangerous:

```
readLock.lock();

try {
    writeLock.lock(); // Can wait indefinitely
} finally {
    readLock.unlock();
}
```

If multiple readers attempt this, every reader can wait for all readers to release their read locks while continuing to hold its own read lock.

A safer pattern is:

```
Release read lock
    ↓
Acquire write lock
    ↓
Recheck the condition
    ↓
Update
```

The state must be checked again because another thread may modify it between releasing the read lock and acquiring the write lock.

---

11. StampedLock

StampedLock supports three modes:

```
Write locking
Read locking
Optimistic reading
```

It returns a long value called a **stamp**.

```
StampedLock lock = new StampedLock();

long stamp = lock.readLock();

try {
    readState();
} finally {
    lock.unlockRead(stamp);
}
```

The correct stamp must be supplied when unlocking.

Important differences

StampedLock:

- Does not implement Lock
- Does not implement ReadWriteLock
- Is not reentrant
- Does not provide Condition
- Requires careful stamp management

Because it is not reentrant, a thread that already holds a stamped lock must not assume it can safely acquire the same mode again.

---

12. Pessimistic read and write with StampedLock

Read lock

```
long stamp = lock.readLock();

try {
    return readValue();
} finally {
    lock.unlockRead(stamp);
}
```

This is a pessimistic read because it actually acquires a shared read lock.

Write lock

```
long stamp = lock.writeLock();
```

```
2
3  try {
4      updateValue();
5  } finally {
6      lock.unlockWrite(stamp);
7  }
```

This is an exclusive write lock.

---

13. Optimistic reading with StampedLock

An optimistic read does not acquire a conventional read lock:

```
1  long stamp = lock.tryOptimisticRead();
```

The thread reads the fields and then validates the stamp:

```
1  double currentX = x;
2  double currentY = y;
3
4  if (!lock.validate(stamp)) {
5      // A writer may have modified the state.
6  }
```

If validation fails, retry using a proper read lock.

Complete example:

```
1  import java.util.concurrent.locks.StampedLock;
2
3  public class Point {
4
5      private final StampedLock lock =
6          new StampedLock();
7
8      private double x;
9      private double y;
10
11      public double distanceFromOrigin() {
12          long stamp = lock.tryOptimisticRead();
13
14          double currentX = x;
15          double currentY = y;
16
```

```
17          if (!lock.validate(stamp)) {
18              stamp = lock.readLock();
19
20              try {
21                  currentX = x;
22                  currentY = y;
23              } finally {
24                  lock.unlockRead(stamp);
25              }
26          }
27
28          return Math.sqrt(
29              currentX * currentX
30                  + currentY * currentY
31          );
32      }
33
34      public void move(double deltaX, double deltaY) {
35          long stamp = lock.writeLock();
36
37          try {
38              x += deltaX;
39              y += deltaY;
40          } finally {
41              lock.unlockWrite(stamp);
42          }
43      }
44  }
```

14. StampedLock optimism versus database optimistic locking

Your description:

> There is no lock, it maintains a version and checks the version before updating.

describes **version-based optimistic locking**, often seen in databases and ORM systems:

```
1  UPDATE account
2  SET balance = ?, version = version + 1
```

```
3  WHERE id = ? AND version = ?
```

If zero rows are updated, another transaction changed the record.

StampedLock.tryOptimisticRead() is related in principle, but its immediate goal is different:

```
1  Obtain an observation stamp
2      ↓
3  Read fields without a read lock
4      ↓
5  Validate that no conflicting write occurred
6      ↓
7  If invalid, retry under a real read lock
```

The stamp is an internal lock-state token. It should not be treated as a business-object version number that application code increments.

Also, optimistic reads require care. You may temporarily observe inconsistent fields before validation. Code should read the fields into local variables, validate, and only use them after successful validation.

---

15. Pessimistic versus optimistic concurrency

Pessimistic locking

Pessimistic locking assumes conflict is sufficiently likely that access should be protected before proceeding.

Examples:

```
1  synchronized
2  ReentrantLock
3  ReentrantReadWriteLock
4  StampedLock.readLock()
5  StampedLock.writeLock()
```

Conceptually:

```
1  Acquire permission first
2      ↓
3  Access state
4      ↓
5  Release permission
```

Optimistic concurrency

Optimistic concurrency assumes conflicts are uncommon.

Conceptually:

```
Observe state
    ↓
Perform work
    ↓
Validate that no conflicting change occurred
    ↓
Accept result or retry
```

Examples include:

- StampedLock.tryOptimisticRead()
- Compare-and-set operations
- Atomic classes
- Database version columns
- JPA @Version

These mechanisms are related, but their exact validation models differ.

---

16. Semaphore

Semaphore is a permit-based synchronizer:

```
Semaphore semaphore = new Semaphore(2);
```

With two permits, up to two threads can enter the controlled region concurrently:

```
semaphore.acquire();

try {
    accessLimitedResource();
} finally {
    semaphore.release();
}
```

Conceptually:

```
Available permits = 2

Thread 1 acquires → permits = 1
Thread 2 acquires → permits = 0
```

```
Thread 3 waits
```

When Thread 1 releases:

```
Thread 1 releases → permit available
Thread 3 acquires
```

Important correction

Semaphore is not a lock implementation:

```
Semaphore does not implement Lock
```

It is a synchronizer that controls concurrency through permits.

---

17. Binary semaphore versus mutex

A semaphore with one permit:

```
Semaphore semaphore = new Semaphore(1);
```

can enforce one-at-a-time access, similar to a mutex.

However, it differs from ReentrantLock:

- A semaphore is not reentrant
- A semaphore has no ownership requirement
- One thread can acquire a permit and another thread can technically release it
- A lock is normally unlocked by its owner
- A semaphore can have multiple permits

Therefore, use:

```
ReentrantLock
```

for ordinary exclusive locking, and:

```
Semaphore
```

to limit concurrent usage of a finite resource.

Typical semaphore use cases:

- Limit concurrent HTTP calls
- Limit access to database connections
- Restrict expensive operations
- Implement application-level bulkheads

---

18. Fair semaphore

A fair semaphore can be created using:

```
1  Semaphore semaphore =
2      new Semaphore(2, true);
```

Fairness generally favors threads in arrival order.

A nonfair semaphore:

```
1  Semaphore semaphore =
2      new Semaphore(2);
```

may allow barging, where a newly arriving thread obtains a permit before an already-waiting thread.

Fairness can reduce throughput, so it should be enabled only when the fairness requirement justifies the cost.

---

19. Permit-leak danger

This is incorrect:

```
1  semaphore.acquire();
2  performWork();
3  semaphore.release();
```

If performWork() throws an exception, the permit is never restored.

Use:

```
1  semaphore.acquire();
2
3  try {
4      performWork();
5  } finally {
6      semaphore.release();
7  }
```

An additional subtlety is that release() should happen only if acquisition succeeded:

```
1  boolean acquired = false;
2
3  try {
4      semaphore.acquire();
5      acquired = true;
```

```
6
7       performWork();
8   } finally {
9       if (acquired) {
10          semaphore.release();
11       }
12  }
```

This matters because acquire() can throw InterruptedException.

---

20. Inter-thread communication with wait() and notify()

The intrinsic monitor methods are:

```
1  wait()
2  notify()
3  notifyAll()
```

They belong to:

```
1  java.lang.Object
```

They must be called while holding that object's monitor:

```
1  synchronized (monitor) {
2      monitor.wait();
3  }
```

This is invalid:

```
1  monitor.wait();
```

without synchronization. It throws:

```
1  IllegalMonitorStateException
```

The same monitor must be used for synchronization and waiting:

```
1  synchronized (monitor) {
2      monitor.wait();
3  }
```

---

21. What does wait() do?

When a thread calls:

```
1  monitor.wait();
```

it:

1. Releases the monitor
2. Enters the monitor's waiting set
3. Remains suspended until notification, interruption, or timeout
4. Competes to reacquire the monitor
5. Returns only after reacquiring the monitor

This release behavior is essential:

```
Thread A holds monitor
    ↓
Condition is false
    ↓
Thread A calls wait()
    ↓
Thread A releases monitor
    ↓
Thread B acquires monitor and changes state
```

If wait() did not release the monitor, Thread B could not enter the synchronized block to make the condition true.

---

22. notify() versus notifyAll()

notify()

```
monitor.notify();
```

Wakes one arbitrary thread waiting on that monitor.

It does not immediately transfer monitor ownership. The notifying thread continues until it exits the synchronized block.

notifyAll()

```
monitor.notifyAll();
```

Wakes all threads waiting on the monitor. They then compete to reacquire it.

notifyAll() is often safer when multiple logical conditions share one monitor because notify() may wake a thread whose condition is still false.

---

23. Always wait in a loop

This is incorrect:

```
synchronized (monitor) {
    if (queue.isEmpty()) {
        monitor.wait();
    }

    consume(queue.remove());
}
```

Use:

```
synchronized (monitor) {
    while (queue.isEmpty()) {
        monitor.wait();
    }

    consume(queue.remove());
}
```

The loop is necessary because:

- Spurious wakeups are permitted
- Another thread may consume the item before this thread reacquires the monitor
- Notification means the state may have changed, not that the required condition is definitely true

The correct discipline is:

```
Wake up
    ↓
Reacquire lock
    ↓
Recheck predicate
    ↓
Proceed only when predicate is true
```

24. Condition

A Condition works with a Lock, commonly ReentrantLock.

```
Lock lock = new ReentrantLock();
```

```
2  Condition condition = lock.newCondition();
```

The rough correspondence is:

```
1  Object monitor API       Lock/Condition API
2  wait()                   await()
3  notify()                 signal()
4  notifyAll()              signalAll()
```

This is a conceptual mapping, not a claim that the implementations are identical.

---

25. Condition.await()

A thread must hold the associated lock before calling await():

```
1  lock.lock();
2
3  try {
4      while (!requiredState()) {
5          condition.await();
6      }
7
8      performWork();
9  } finally {
10      lock.unlock();
11  }
```

await():

1. Atomically releases the associated lock
2. Suspends the thread
3. Waits for a signal, interruption, or timeout
4. Reacquires the lock before returning

Just like wait(), await() should normally be used inside a while loop.

---

26. signal() and signalAll()

The signalling thread must hold the associated lock:

```
1  lock.lock();
2
3  try {
```

```
        changeSharedState();
        condition.signal();
    } finally {
        lock.unlock();
    }
```

signal() wakes one waiting thread.

```
condition.signal();
```

signalAll() wakes all waiting threads:

```
condition.signalAll();
```

A signalled thread does not immediately execute the protected section. It must first reacquire the lock.

---

27. Why Condition is more flexible

An intrinsic monitor has one wait set:

```
One monitor
    ↓
One logical waiting set
```

A ReentrantLock can create multiple Condition objects:

```
Condition notEmpty = lock.newCondition();
Condition notFull = lock.newCondition();
```

This lets the application wake only the relevant category of waiting threads:

```
Consumers wait on notEmpty
Producers wait on notFull
```

This is more precise than waking every thread through one monitor's notifyAll().

---

28. Bounded buffer using Condition

```
import java.util.ArrayDeque;
import java.util.Queue;
import java.util.concurrent.locks.Condition;
import java.util.concurrent.locks.Lock;
import java.util.concurrent.locks.ReentrantLock;

public class BoundedBuffer<T> {
```

```
8
9       private final Queue<T> queue =
10          new ArrayDeque<>();
11
12      private final int capacity;
13
14      private final Lock lock =
15          new ReentrantLock();
16
17      private final Condition notEmpty =
18          lock.newCondition();
19
20      private final Condition notFull =
21          lock.newCondition();
22
23      public BoundedBuffer(int capacity) {
24          if (capacity <= 0) {
25              throw new IllegalArgumentException(
26                  "Capacity must be positive"
27              );
28          }
29
30          this.capacity = capacity;
31      }
32
33      public void put(T value)
34              throws InterruptedException {
35
36          lock.lockInterruptibly();
37
38          try {
39              while (queue.size() == capacity) {
40                  notFull.await();
41              }
42
43              queue.add(value);
44
45              notEmpty.signal();
46          } finally {
47              lock.unlock();
```

```
48          }
49      }
50
51      public T take()
52              throws InterruptedException {
53
54          lock.lockInterruptibly();
55
56          try {
57              while (queue.isEmpty()) {
58                  notEmpty.await();
59              }
60
61              T value = queue.remove();
62
63              notFull.signal();
64
65              return value;
66          } finally {
67              lock.unlock();
68          }
69      }
70  }
```

Producer behavior

```
1  Acquire lock
2      ↓
3  Queue full?
4      ├── Yes → await on notFull
5      └── No  → insert item
6                        ↓
7              signal notEmpty
```

Consumer behavior

```
1  Acquire lock
2      ↓
3  Queue empty?
4      ├── Yes → await on notEmpty
5      └── No  → remove item
6                        ↓
```

```
7                 signal notFull
```

For production code, use an existing BlockingQueue unless implementing the mechanism is itself the requirement:

```
1  BlockingQueue<T> queue =
2      new ArrayBlockingQueue<>(capacity);
```

29. await() variations

Condition offers more waiting options than basic Object.wait():

```
1  condition.await();
2  condition.awaitUninterruptibly();
3  condition.awaitNanos(nanos);
4  condition.await(time, unit);
5  condition.awaitUntil(deadline);
```

Example with timeout:

```
1  boolean available =
2      condition.await(2, TimeUnit.SECONDS);
3
4  if (!available) {
5      // Timed out
6  }
```

The shared-state predicate must still be rechecked.

30. Choosing the correct mechanism

Use synchronized when

- Simple exclusive mutual exclusion is enough
- One waiting condition is sufficient
- Timed or interruptible lock acquisition is unnecessary
- Simpler code is preferred

Use ReentrantLock when

- You need tryLock()
- You need timed lock acquisition
- Lock acquisition must be interruptible

- You need multiple Condition objects
- Explicit fairness is required

Use ReentrantReadWriteLock when

- Reads greatly outnumber writes
- Read operations can safely execute concurrently
- Contention is measurable
- Benchmarking demonstrates a benefit

Use StampedLock when

- Read-heavy access needs optimistic reads
- The protected data can be copied into local variables and validated
- Non-reentrancy and manual stamp management are acceptable
- Measurements justify the added complexity

Use Semaphore when

- Up to N threads may access a limited resource
- You are controlling concurrency rather than protecting one mutual-exclusion invariant
- Examples include connection, request, and rate limits

Use Condition when

- Threads must wait for a state predicate
- You are using an explicit Lock
- Multiple logical wait sets are helpful

---

31. Corrected summary

```
synchronized
    → Locks a monitor object
    → Can coordinate different instances if they use the same monitor
```

```
ReentrantLock
    → Exclusive, explicit and reentrant lock
    → Supports tryLock, timeouts, interruption and Conditions
```

```
Read lock
    → Shared
    → Multiple readers can coexist
```

```
Write lock
    → Exclusive
    → Blocks readers and other writers
```

```
StampedLock
    → Supports read, write and optimistic-read modes
    → Uses stamps
    → Is not reentrant
```

```
Semaphore
    → Permit-based concurrency limiter
    → Is not a Lock implementation
```

```
wait / notify / notifyAll
    → Used while owning an intrinsic monitor
```

```
await / signal / signalAll
    → Used while holding the associated explicit Lock
```

```
wait() and await()
    → Release their associated monitor or lock while waiting
    → Reacquire it before returning
    → Must normally be used in a while loop
```

Main improvements to your notes

Technical accuracy

- synchronized **works across different objects when they share the same monitor.**
- Semaphore is a permit-based synchronizer, not an implementation of Lock.
- StampedLock optimistic reading validates a stamp and is not the same mechanism as database version-column locking.
- wait() and await() release the relevant monitor or lock while waiting.
- signal() is similar to notify(), while signalAll() corresponds conceptually to notifyAll().

Design clarity

The central rule is not “custom locks do not depend on objects.” Every explicit lock is still an object. The correct rule is:

> Every thread accessing the same shared invariant must coordinate through the same monitor, lock, semaphore, or other synchronization mechanism.